

Importing the Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score
from sklearn.model_selection import train_test_split
import pickle
import csv
```

Defining the Dataset link= https://drive.google.com/file/d/1yVwfX1LjISQ6vdR1Kaht-S1_8y2Gk6z/view?usp=sharing

```
df = pd.read_csv('Housing.csv')
print(df.to_string())
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	yes
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	no
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	yes
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	yes
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	no
5	10850000	7500	3	3	1	yes	no	yes	no	yes	2	yes
6	10150000	8580	4	3	4	yes	no	no	no	yes	2	yes
7	10150000	16200	5	3	2	yes	no	no	no	no	0	no
8	9870000	8100	4	1	2	yes	yes	yes	no	yes	2	yes
9	9800000	5750	3	2	4	yes	yes	no	no	yes	1	yes
10	9800000	13200	3	1	2	yes	no	yes	no	yes	2	yes
11	9681000	6000	4	3	2	yes	yes	yes	yes	no	2	no
12	9310000	6550	4	2	2	yes	no	no	no	yes	1	yes
13	9240000	3500	4	2	2	yes	no	no	yes	no	2	no
14	9240000	7800	3	2	2	yes	no	no	no	no	0	yes
15	9100000	6000	4	1	2	yes	no	yes	no	no	2	no
16	9100000	6600	4	2	2	yes	yes	yes	no	yes	1	yes
17	8960000	8500	3	2	4	yes	no	no	no	yes	2	no
18	8890000	4600	3	2	2	yes	yes	no	no	yes	2	no
19	8855000	6420	3	2	2	yes	no	no	no	yes	1	yes
20	8750000	4320	3	1	2	yes	no	yes	yes	no	2	no
21	8680000	7155	3	2	1	yes	yes	yes	no	yes	2	no
22	8645000	8050	3	1	1	yes	yes	yes	no	yes	1	no
23	8645000	4560	3	2	2	yes	yes	yes	no	yes	1	no
24	8575000	8800	3	2	2	yes	no	no	no	yes	2	no
25	8540000	6540	4	2	2	yes	yes	yes	no	yes	2	yes
26	8463000	6000	3	2	4	yes	yes	yes	no	yes	0	yes
27	8400000	8875	3	1	1	yes	no	no	no	no	1	no
28	8400000	7950	5	2	2	yes	no	yes	yes	no	2	no
29	8400000	5500	4	2	2	yes	no	yes	no	yes	1	yes
30	8400000	7475	3	2	4	yes	no	no	no	yes	2	no
31	8400000	7000	3	1	4	yes	no	no	no	yes	2	no
32	8295000	4880	4	2	2	yes	no	no	no	yes	1	yes
33	8190000	5960	3	3	2	yes	yes	yes	no	no	1	no
34	8120000	6840	5	1	2	yes	yes	yes	no	yes	1	no
35	8080940	7000	3	2	4	yes	no	no	no	yes	2	no
36	8043000	7482	3	2	3	yes	no	no	yes	no	1	yes
37	7980000	9000	4	2	4	yes	no	no	no	yes	2	no
38	7962500	6000	3	1	4	yes	yes	no	no	yes	2	no
39	7910000	6000	4	2	4	yes	no	no	no	yes	1	no
40	7875000	6550	3	1	2	yes	no	yes	no	yes	0	yes
41	7840000	6360	3	2	4	yes	no	no	no	yes	0	yes
42	7700000	6480	3	2	4	yes	no	no	no	yes	2	no
43	7700000	6000	4	2	4	yes	no	no	no	no	2	no
44	7560000	6000	4	2	4	yes	no	no	no	yes	1	no
45	7560000	6000	3	2	3	yes	no	no	no	yes	0	no
46	7525000	6000	3	2	4	yes	no	no	no	yes	1	no
47	7490000	6600	3	1	4	yes	no	no	no	yes	3	yes
48	7455000	4300	3	2	2	yes	no	yes	no	no	1	no
49	7420000	7440	3	2	1	yes	yes	yes	no	yes	0	yes
50	7420000	7440	3	2	4	yes	no	no	no	no	1	yes
51	7420000	6325	3	1	4	yes	no	no	no	yes	1	no
52	7350000	6000	4	2	4	yes	yes	no	no	yes	1	no
53	7350000	5150	3	2	4	yes	no	no	no	yes	2	no
54	7350000	6000	3	2	2	yes	yes	no	no	yes	1	no
55	7350000	6000	3	1	2	yes	no	no	no	yes	1	no

EDA- Exploratory Data Analysis

```
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwa
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	ves	

df.tail()

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwa
540	1820000	3000	2	1	1	yes	no	yes	
541	1767150	2400	3	1	1	no	no	no	
542	1750000	3620	2	1	1	yes	no	no	
543	1750000	2910	3	1	1	no	no	no	
544	1750000	3850	3	1	2	ves	no	no	

df.shape[0]

545

df.shape[1]

13

df.shape

(545, 13)

df.columns

Index(['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad',
 'guestroom', 'basement', 'hotwaterheating', 'airconditioning',
 'parking', 'prefarea', 'furnishingstatus'],
 dtype='object')

len(df.columns)

13

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
Column Non-Null Count Dtype
--- ---
0 price 545 non-null int64
1 area 545 non-null int64
2 bedrooms 545 non-null int64
3 bathrooms 545 non-null int64
4 stories 545 non-null int64
5 mainroad 545 non-null object
6 guestroom 545 non-null object
7 basement 545 non-null object
8 hotwaterheating 545 non-null object
9 airconditioning 545 non-null object
10 parking 545 non-null int64
11 prefarea 545 non-null object
12 furnishingstatus 545 non-null object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB

df.describe()

	price	area	bedrooms	bathrooms	stories	parking
count	5.450000e+02	545.000000	545.000000	545.000000	545.000000	545.000000
mean	4.766729e+06	5150.541284	2.965138	1.286239	1.805505	0.693578
std	1.870440e+06	2170.141023	0.738064	0.502470	0.867492	0.861586
min	1.750000e+06	1650.000000	1.000000	1.000000	1.000000	0.000000
25%	3.430000e+06	3600.000000	2.000000	1.000000	1.000000	0.000000

df.dtypes	
price	int64
area	int64
bedrooms	int64
bathrooms	int64
stories	int64
mainroad	object
guestroom	object
basement	object
hotwaterheating	object
airconditioning	object
parking	int64
prefarea	object
furnishingstatus	object
dtype: object	

df.nunique()	
price	219
area	284
bedrooms	6
bathrooms	4
stories	4
mainroad	2
guestroom	2
basement	2
hotwaterheating	2
airconditioning	2
parking	4
prefarea	2
furnishingstatus	3
dtype: int64	

df.isna().sum()	
price	0
area	0
bedrooms	0
bathrooms	0
stories	0
mainroad	0
guestroom	0
basement	0
hotwaterheating	0
airconditioning	0
parking	0
prefarea	0
furnishingstatus	0
dtype: int64	

df.isna().sum().sum()	
0	

sns.boxplot(df.price)	
-----------------------	--



```
IQR= df.price.quantile(0.75) - df.price.quantile(0.25)
IQR
```

2310000.0

```
lower_limit = df.price.quantile(0.25) - 1.5*IQR
upper_limit = df.price.quantile(0.75) - 1.5*IQR
```

lower_limit

-35000.0

upper_limit

2275000.0

```
df.price.describe()
```

```
count    5.450000e+02
mean      4.766729e+06
std       1.870440e+06
min       1.750000e+06
25%       3.430000e+06
50%       4.340000e+06
75%       5.740000e+06
max       1.330000e+07
Name: price, dtype: float64
```

```
# Finding the values that are outside the upper bound
df[df["price"]>upper_limit]
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hot
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	
...	...	...	...	...	...	...	...	...	
522	2380000	2475	3	1	2	yes	no	no	
523	2380000	2787	4	2	2	yes	no	no	
524	2380000	3264	2	1	1	yes	no	no	
525	2345000	3640	2	1	1	yes	no	no	
526	2310000	3180	2	1	1	yes	no	no	

527 rows x 13 columns

```
len(df)
```

545

```
545-527
```

18

```
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwater
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	

```
df.mainroad.unique()
```

```
array(['yes', 'no'], dtype=object)
```

```
#yes -> 1
#no -> 0
df.mainroad.replace({'yes','no'},[1,0])

0      1
1      1
2      1
3      1
4      1
..
540    1
541    0
542    1
543    0
544    1
Name: mainroad, Length: 545, dtype: int64
```

```
newdf = df.copy()
```

```
enc = LabelEncoder()
```

```
newdf.mainroad
```

```
0      yes
1      yes
2      yes
3      yes
4      yes
...
540    yes
541     no
542    yes
543     no
544    yes
Name: mainroad, Length: 545, dtype: object
```

```
newdf.mainroad = enc.fit_transform(newdf.mainroad)
```

```
newdf.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwater
0	13300000	7420	4	2	3	1	no	no	
1	12250000	8960	4	4	4	1	no	no	
2	12250000	9960	3	2	2	1	no	yes	
3	12215000	7500	4	2	2	1	no	yes	
4	11410000	7420	4	1	2	1	yes	yes	

```
newdf.mainroad.unique()
```

```
array([1, 0])
```

```
newdf.basement.unique()
```

```
array(['no', 'yes'], dtype=object)
```

```
newdf.basement.value_counts()
```

```
no      354
yes     191
Name: basement, dtype: int64
```



```
0      140
Name: furnishingstatus, dtype: int64
```

```
1 -> semi-furnished
2 -> unfurnished
3 -> furnished
```

- 1. Data Strenght - 545
- 2. Number of Random Variables - 13
- 3. None of the columns contained Null values
- 4. Columns- price,area,bedroom,bathroom,stories & parking has numerical values
- 5. Columns- mainroad,guestroom,hotwaterheating,airconditioning,preface & furnishing status has objects values

```
len(df)
```

545

```
len(df.columns)
```

13

```
df.dtypes
```

```
price          int64
area           int64
bedrooms       int64
bathrooms      int64
stories        int64
mainroad       int64
guestroom      int64
basement       int64
hotwaterheating int64
airconditioning int64
parking        int64
prefarea       int64
furnishingstatus int64
dtype: object
```

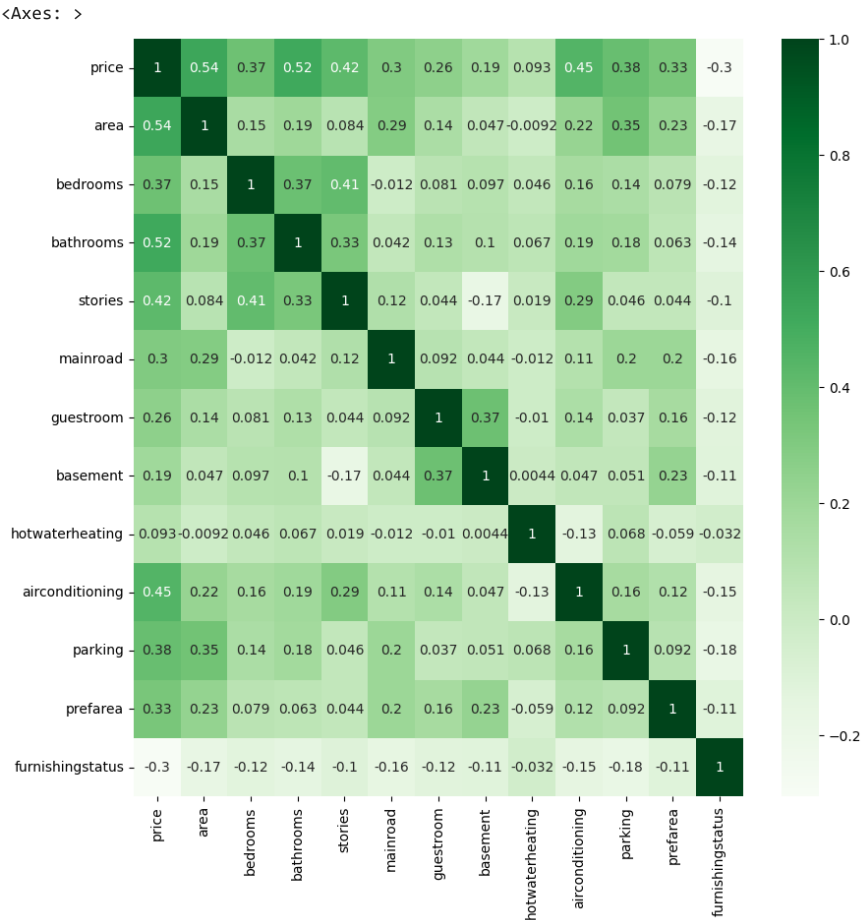
```
df.describe()
```

	price	area	bedrooms	bathrooms	stories	mainroad	gues
count	5.450000e+02	545.000000	545.000000	545.000000	545.000000	545.000000	545.0
mean	4.766729e+06	5150.541284	2.965138	1.286239	1.805505	0.858716	0.1
std	1.870440e+06	2170.141023	0.738064	0.502470	0.867492	0.348635	0.3
min	1.750000e+06	1650.000000	1.000000	1.000000	1.000000	0.000000	0.0
25%	3.430000e+06	3600.000000	2.000000	1.000000	1.000000	1.000000	0.0
50%	4.340000e+06	4600.000000	3.000000	1.000000	2.000000	1.000000	0.0
75%	5.740000e+06	6360.000000	3.000000	2.000000	2.000000	1.000000	0.0
max	1.330000e+07	16200.000000	6.000000	4.000000	4.000000	1.000000	1.0

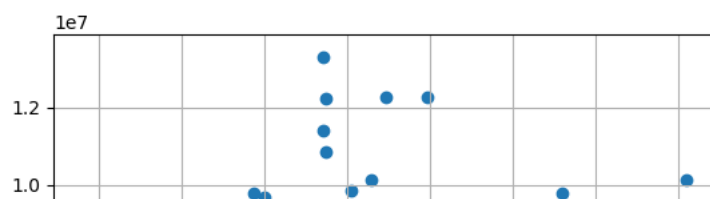
```
df.corr()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestro
price	1.000000	0.535997	0.366494	0.517545	0.420712	0.296898	0.2555
area	0.535997	1.000000	0.151858	0.193820	0.083996	0.288874	0.1402
bedrooms	0.366494	0.151858	1.000000	0.373930	0.408564	-0.012033	0.0805

```
plt.figure(figsize=(10,10))
sns.heatmap(df.corr(), annot=True, cmap='Greens')
```



```
plt.scatter(df.area, df.price)
plt.xlabel('Area')
plt.ylabel('Price')
plt.grid()
```

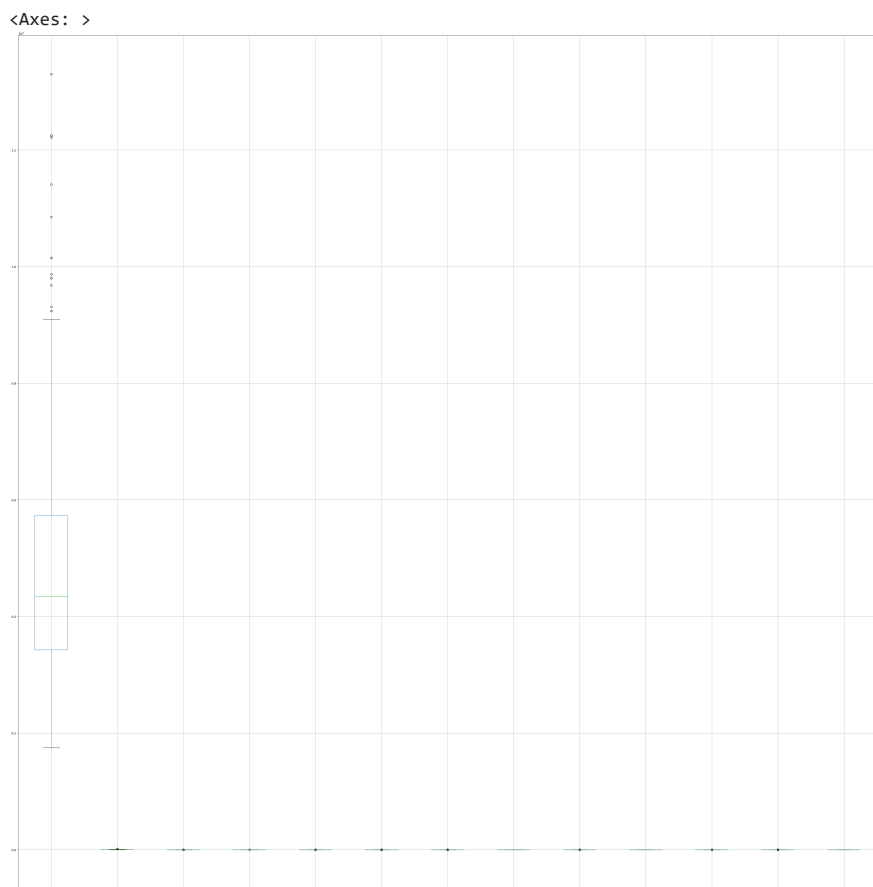
```
sns.pairplot(df)
```

```
df.bedrooms.unique()
```

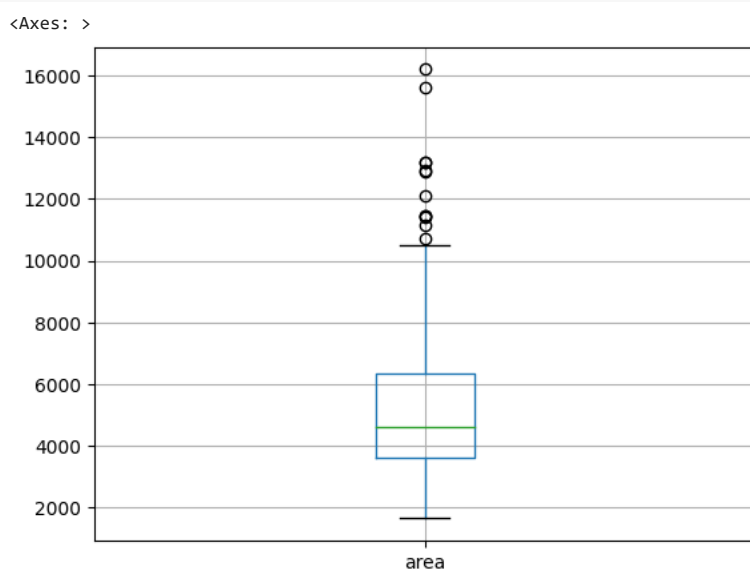
```
array([4, 3, 5, 2, 6, 1])
```

```
plt.figure(figsize=(50,50))
```

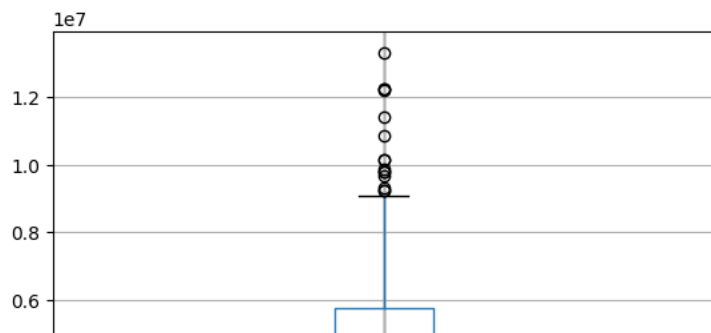
```
df.boxplot()
```



```
df[['area']].boxplot()
```

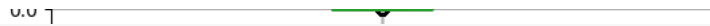


```
for i in df.columns:
    df[[f'{i}']].boxplot()
plt.plot()
```



```
df.furnishingstatus.value_counts()
```

```
1    227
2    178
0    140
Name: furnishingstatus, dtype: int64
```

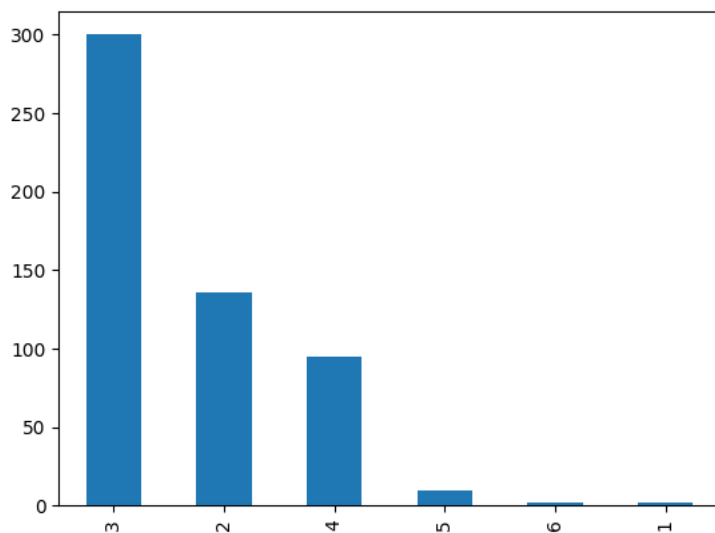


```
#How many values are there where bedrooms is more than 4
df.bedrooms[df.bedrooms >=4].count()
```

```
107
```

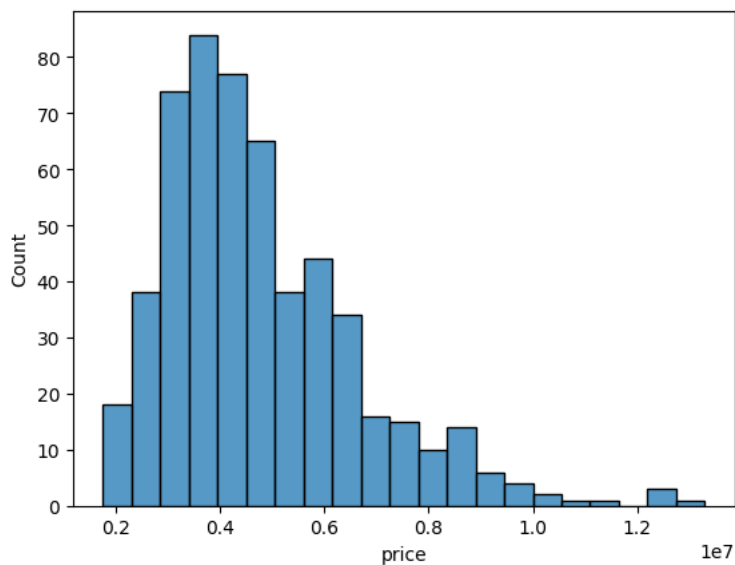
```
df.bedrooms.value_counts().plot.bar()
```

<Axes: >



```
sns.histplot(df.price)
```

<Axes: xlabel='price', ylabel='Count'>



```
#What is the price of the house where bedrooms are 4 and bathrooms is 2
df[['price', 'bedrooms', 'bathrooms']][df.bedrooms==4 & (df.bathrooms == 2)]
```

```
len(df[['price', 'bedrooms', 'bathrooms']][df.bedrooms==4 & (df.bathrooms==2)])
```

48

Machine Learning

```
#Defining the model
model = LinearRegression()
```

```
#Defining the independent & dependent variable
df.head(2)
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwa
0	13300000	7420	4	2	3	1	0	0	
1	12250000	8960	4	4	4	1	0	0	

```
x = df.iloc[:,1:]
y = df.price
```

x

	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheati
0	7420	4	2	3	1	0	0	
1	8960	4	4	4	1	0	0	
2	9960	3	2	2	1	0	1	
3	7500	4	2	2	1	0	1	
4	7420	4	1	2	1	1	1	
...	...	...	...	...	...	...	...	
540	3000	2	1	1	1	0	1	
541	2400	3	1	1	0	0	0	
542	3620	2	1	1	1	0	0	
543	2910	3	1	1	0	0	0	
544	3850	3	1	2	1	0	0	

545 rows x 12 columns

Train Test Split

```
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.2, random_state=42)
```

69 6790000 4 2

```
len(x_train)
```

436

```
len(x_test)
```

109

```
#Training the model
model.fit(x_train, y_train)
```

LinearRegression
LinearRegression()

142 5600000 4 2

```
model.coef_
```

```
array([ 2.35848772e+02,  7.85744924e+04,  1.09711727e+06,  4.06223164e+05,
        3.66824192e+05,  2.33146766e+05,  3.93159779e+05,  6.87881311e+05,
        7.85550579e+05,  2.25756514e+05,  6.29901661e+05, -2.10397118e+05])
```

```
#Cost Function (Intercept Value)
model.intercept_
```

293083.06915066857

y = c+ m1x1 + m2x2 +m3x3 +....+ m(n)x(n)

Testing/Prediction

```
y_pred = model.predict(x_test)
```

```
y_pred
```

```
array([5203691.70963178, 7257004.02115475, 3062828.59668171,
       4559591.65374424, 3332932.30559782, 3563080.67918997,
       5645466.31219972, 6413979.66873635, 2755831.54819 ,
       2668938.66075228, 9570600.29915353, 2827431.50860062,
       3195686.2583409 , 3352263.99438471, 3713879.49996131,
       5301088.24435749, 2987920.2666968 , 4810799.8212371 ,
       4383031.70489929, 3525092.18938646, 5796259.50068013,
       5840000.70299301, 2760214.608641 , 4762590.14920608,
       5204755.73895206, 7515542.71619025, 3254681.68956382,
       5236164.45964444, 8178523.16820284, 3434166.15675649,
       6443921.58767581, 3346004.77919184, 6742324.74004133,
       4154936.84088665, 3589152.47491253, 5788125.92515322,
       4768370.18154076, 4391684.04193171, 3217657.04549935,
```

```
4638196.61928878, 4522160.27786714, 3541284.06127245,
7238136.1194117 , 4021515.68926614, 3701978.76822757,
4298879.55563098, 6705004.0206061 , 3993466.52296896,
3798185.05328059, 3451821.5624289 , 7293996.86867461,
2832905.26972678, 4378698.1476993 , 4468002.85418353,
3714623.89765883, 2718466.90282779, 7524449.64983626,
2950437.1495281 , 4194596.28592921, 2795827.77318101,
4801788.72577028, 3618230.29875972, 5091685.77019752,
4248122.70647651, 4729964.20281946, 4620982.9363411 ,
7215846.76983927, 3485106.19132682, 5933820.14349914,
6236580.98095547, 4809923.61015999, 5130920.22938947,
4527282.87392583, 8006869.07208366, 3543927.63713715,
5463086.05281816, 3902441.28505163, 4661063.63495886,
4825370.89331733, 4276803.47099967, 7754072.71344458,
4008573.23235629, 6500977.91345037, 5386796.00909808,
2785343.0092764 , 6836356.36771845, 2638553.49378047,
3634231.99215348, 8004072.41154001, 8048399.75418243,
3227573.69687308, 5935167.4198354 , 3605772.48317454,
3817767.91420862, 7631673.34074558, 4775106.62670537,
5093491.44442931, 6447437.14797692, 4943449.6813564 ,
5943929.14295426, 3810460.26405425, 6457426.32302761,
3701491.23218178, 5960492.47343073, 4933137.14530295,
4377133.13624287, 7009335.02696999, 6351460.97128317,
6409340.41089671])
```

y_test

```
316    4060000
77      6650000
360     3710000
90      6440000
493     2800000
...
15      9100000
357     3773000
39      7910000
54      7350000
155     5530000
Name: price, Length: 109, dtype: int64
```

```
newdf = pd.DataFrame({"Actual value": y_test, "Predicted value": y_pred})
```

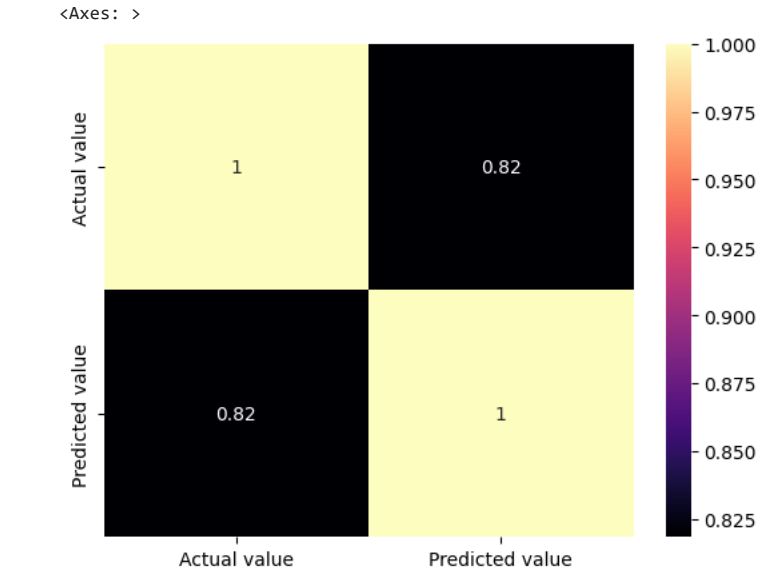
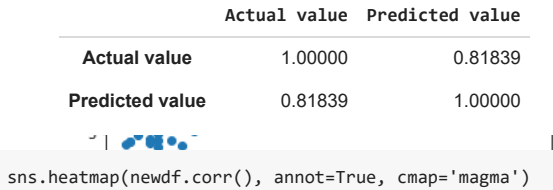
newdf

	Actual value	Predicted value
316	4060000	5.203692e+06
77	6650000	7.257004e+06
360	3710000	3.062829e+06
90	6440000	4.559592e+06
493	2800000	3.332932e+06
...	...	...
15	9100000	4.933137e+06
357	3773000	4.377133e+06
39	7910000	7.009335e+06
54	7350000	6.351461e+06
155	5530000	6.409340e+06

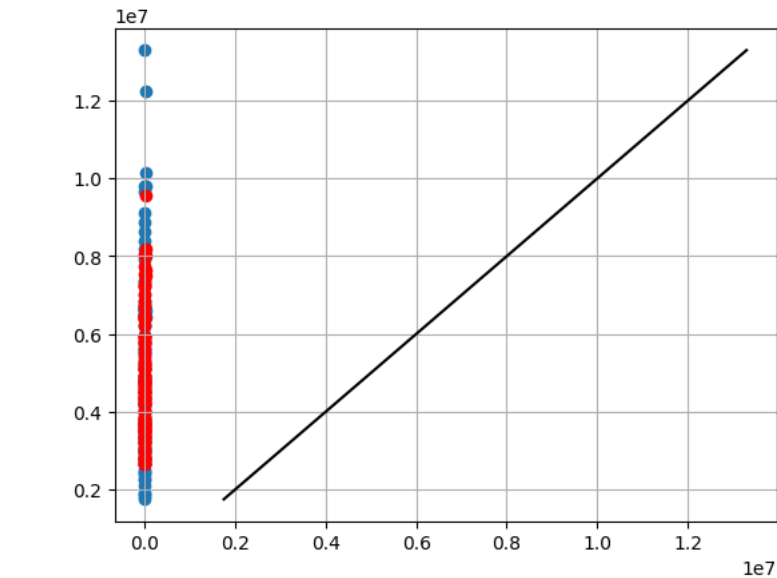
109 rows × 2 columns

```
plt.scatter(newdf['Actual Value'], newdf['Predicted Value'])
plt.xlabel('Actual Value')
plt.ylabel('Predicted Value')
plt.title('Actual V/s Predicted')
```

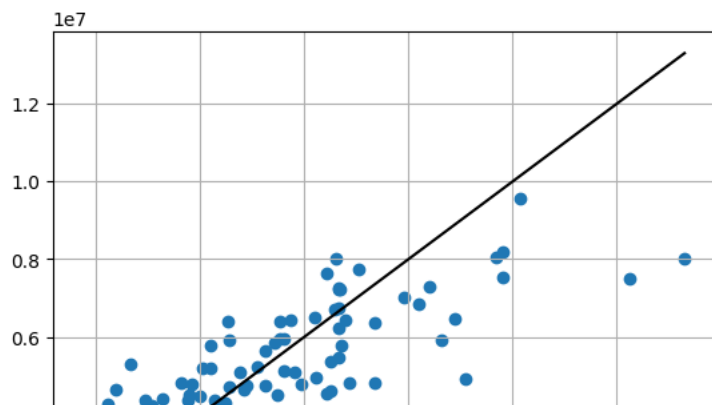
```
Text(0.5, 1.0, 'Actual V/s Predicted')
newdf.corr()
```



```
plt.scatter(x_test.area, y_test)
plt.scatter(x_test.area, y_pred, color='r')
plt.plot(y,y,c='black')
plt.grid()
```



```
plt.scatter(y_test, y_pred)
plt.plot(y,y, c='black')
plt.grid()
```



Performance

```
r2_score(y_test, y_pred)
```

```
0.6494754192267803
```

```
model.predict([[2400, 2,2,2,2,2,1,1,1,1,1]])[0]
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:439: UserWarning: X does not have valid feature names, but LinearRegression
warnings.warn(
8127904.387731913
```

Exporting the Model

```
with open('model.pkl','wb') as files:
    pickle.dump(model, files)
```

df

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hoti
0	13300000	7420	4	2	3	1	0	0	
1	12250000	8960	4	4	4	1	0	0	
2	12250000	9960	3	2	2	1	0	1	
3	12215000	7500	4	2	2	1	0	1	
4	11410000	7420	4	1	2	1	1	1	
...	...	...	...	...	...	...	...	...	
540	1820000	3000	2	1	1	1	0	1	
541	1767150	2400	3	1	1	0	0	0	
542	1750000	3620	2	1	1	1	0	0	
543	1750000	2910	3	1	1	0	0	0	
544	1750000	3850	3	1	2	1	0	0	

545 rows x 13 columns

```
def norm_fun(i):
    x = (i-i.min())/(i.max() - i.min())
    return x
```

```
df_normalize = norm_fun(df.iloc[:,1:])
```

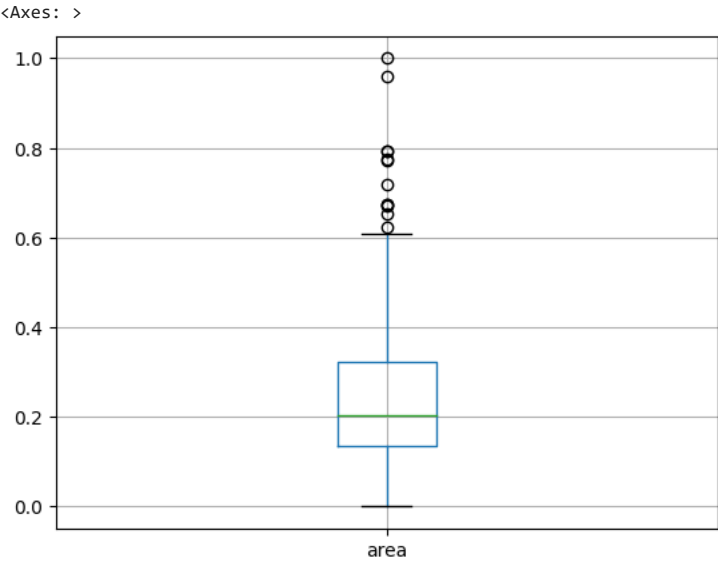
df_normalize

	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterh
0	0.396564	0.6	0.333333	0.666667	1.0	0.0	0.0	
1	0.502405	0.6	1.000000	1.000000	1.0	0.0	0.0	
2	0.571134	0.4	0.333333	0.333333	1.0	0.0	1.0	
3	0.402062	0.6	0.333333	0.333333	1.0	0.0	1.0	
4	0.396564	0.6	0.000000	0.333333	1.0	1.0	1.0	
...	...	...	...	...	...	...	...	
540	0.092784	0.2	0.000000	0.000000	1.0	0.0	1.0	

```
df_normalize.describe()
```

	area	bedrooms	bathrooms	stories	mainroad	guestroom	basemen
count	545.000000	545.000000	545.000000	545.000000	545.000000	545.000000	545.000000
mean	0.240587	0.393028	0.095413	0.268502	0.858716	0.177982	0.35045
std	0.149151	0.147613	0.167490	0.289164	0.348635	0.382849	0.47755
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
25%	0.134021	0.200000	0.000000	0.000000	1.000000	0.000000	0.00000
50%	0.202749	0.400000	0.000000	0.333333	1.000000	0.000000	0.00000
75%	0.323711	0.400000	0.333333	0.333333	1.000000	0.000000	1.00000
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000

```
df_normalize[['area']].boxplot()
```



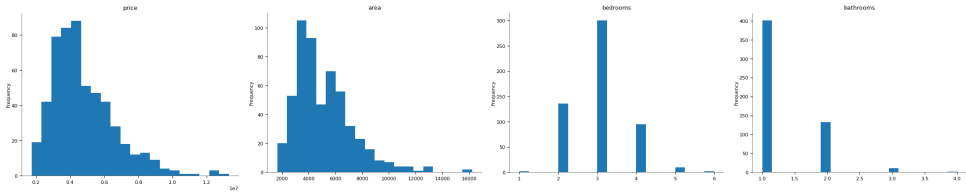
```
df
```



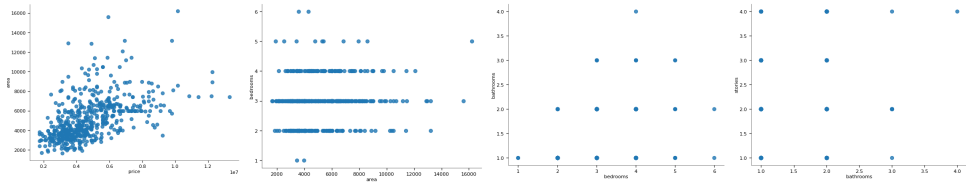
	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefare:
0	13300000	7420	4	2	3	1	0	0	0	1	2	'
1	12250000	8960	4	4	4	1	0	0	0	1	3	(
2	12250000	9960	3	2	2	1	0	1	0	0	2	'
3	12215000	7500	4	2	2	1	0	1	0	1	3	'
4	11410000	7420	4	1	2	1	1	1	0	1	2	(
...	...	...	...	...	...	...	...	...	...	...	...	..
540	1820000	3000	2	1	1	1	0	1	0	0	2	(
541	1767150	2400	3	1	1	0	0	0	0	0	0	(
542	1750000	3620	2	1	1	1	0	0	0	0	0	(
543	1750000	2910	3	1	1	0	0	0	0	0	0	(
544	1750000	3850	3	1	2	1	0	0	0	0	0	(

545 rows × 13 columns

Distributions



2-d distributions



Values